

Spring Boot Web Annotations

What Are Annotations in Spring?

Annotations in Spring are **instructions** to the framework.

They tell Spring:

- Which class is a controller
- Which method handles a request
- How data should be extracted from the request
- How the response should be built
- How objects should be created and injected

Without annotations, Spring would not know how to connect:

HTTP request → Java method → Database → HTTP response

How a Request Enters Spring (Big Picture)

When a user hits: **GET /users/10**

The flow is:

```
Client
↓
Tomcat Server
↓
DispatcherServlet (Front Controller)
↓
HandlerMapping
↓
Controller Method
↓
Service
↓
Repository
↓
Database
↓
Response back to client
```

DispatcherServlet is the brain of Spring MVC.

It looks for:

- Controller classes
- Mapping annotations
- HTTP methods
- URL patterns

@Controller & @RequestMapping

```
@Controller
@RequestMapping("/users")
public class UserController {
}
```

@Controller tells Spring:

This class contains request-handling methods.

@RequestMapping("/users") tells Spring:

All methods inside this controller start with /users

Mapping HTTP Requests

Old style

```
@RequestMapping(value="/{id}", method=RequestMethod.GET)
@ResponseBody
public UserDTO getUser(@PathVariable Long id) {
    return service.getUser(id);
}
```

@ResponseBody tells Spring:

Convert return value into JSON

@RestController

Because @ResponseBody is needed on every method, Spring introduced:

```
@RestController
@RequestMapping("/users")
public class UserController {
}
```

@RestController = @Controller + @ResponseBody

Now everything automatically returns JSON.

@GetMapping, @PostMapping, @PutMapping, @DeleteMapping

Instead of writing:

```
@RequestMapping(value="/{id}", method=RequestMethod.GET)
```

You write:

```
@GetMapping("/{id}")
```

Same for:

- @PostMapping
- @PutMapping
- @DeleteMapping

These are just **specialized RequestMapping** annotations.

@RequestParam vs @PathVariable

Query Parameter

/users?id=10

```
@GetMapping
public UserDTO getUser(@RequestParam Long id) { }
```

Path Variable

/users/10

```
@GetMapping("/{id}")
public UserDTO getUser(@PathVariable Long id) { }
```

@RequestBody

For POST / PUT APIs:

```
@PostMapping
public UserDTO createUser(@RequestBody UserDTO dto) { }
```

Spring:

- Reads JSON
- Converts it into Java object

@Autowired (Dependency Injection)

```
@Autowired
private UserService userService;
```

Spring injects:

- Service into Controller
- Repository into Service

You never create objects with new.

@Component, @Service, @Repository

Annotation	Purpose
@Component	Generic bean
@Service	Business logic
@Repository	Database access + exception translation

Spring uses these to apply:

- Transactions
- Persistence handling
- AOP

Interview-Critical Questions

Q1. Difference between @Controller and @RestController?

@RestController always returns JSON.

Q2. Difference between @RequestParam and @PathVariable?

Query string vs URL path.

Q3. Who handles all incoming requests?

DispatcherServlet.

Q4. Why @Repository instead of @Component?

Exception translation.

One-Line Summary

“Spring MVC annotations map HTTP requests to Java methods and wire controllers, services, and repositories together.”